COMMUNITY HEALTH TECHNOLOGY · LEADERSHIP BRIEF

Strategic Plan: Safer AWS Accounts, Cleaner Repos, Mobile & Self-Hosted Video

A plain-language plan for safer AWS accounts, cleaner repos, mobile, and self-hosted video — and how that model **lowers cost** by scaling each environment to what it needs, while giving teams more autonomy to build robust technology products for the company.

Audience	CHM / CHT leadership, product, operations, compliance (non-technical and technical)
Scope	All CHT product systems: Platform, Content Hub, Companion, Reports
Status	Draft for discussion v1.2
Date	October 2026
Classification	Confidential — internal use

01 Why this document exists

Today, our products run well enough for beta and early production use. The choices below determine whether we can grow without outages, surprise bills, or blocked product work — and whether teams can move fast with clear ownership:

1. **Where we run experiments** — today, development and production share one AWS account.
2. **How our code is organized** — we will split each service into separate application and infrastructure repositories, plus a shared foundations repo.
3. **How people will use CHT on phones** — we are web-first today; mobile needs a deliberate plan.
4. **Where video lives and how we measure it** — we rely heavily on YouTube today; next we add a dedicated video upload / ingest service, a transcript–summary–tags pipeline, and keep Content Hub focused on metadata and analytics.
5. **How we spend and how teams work** — scale each environment down to what it needs, and give product teams autonomy to own and ship robust technology.

> **How to read this**
>
> Each section answers three questions: *What is true today? · Why does it matter? · What do we recommend, in what order?* Technical detail is intentionally light. Implementation specs can follow once leadership agrees on direction.

Contents

02 Separate AWS accounts (Dev, Test / UAT, Production)

03 Mobile app plan

04 Splitting every service into app + infra repos (plus shared foundations)

05 Video services: upload / ingest, transcripts & tags, Content Hub analytics

06 Lower cost, more autonomy, stronger products

07 Suggested sequence & decisions needed

02 Why we need separate AWS accounts (Dev, Test / UAT, Production)

What is true today

We only have two live environments — developer testing (**devapp**) and production (**app**) — and they both run inside **one AWS account**. Cost controls, security alarms, container images, and permissions for every product (Platform, Content Hub, Companion, Reports) share that same account. We are **not** bringing back a “staging” environment; the standard going forward is exactly three names: **Dev**, **Test / UAT**, and **Production**.

What that feels like day to day

- A mistaken change in “dev” can affect production resources more easily.
- Developers share the same billing and permission boundary as live users.
- Turning things off overnight to save money is harder and riskier.
- There is no safe place for stakeholders to sign off before a production release.

Analogy

It is like using the same house key for the workshop, the dress rehearsal room, and the patient’s real clinic. You can be careful — but one wrong turn still opens the wrong door.

Our three environments (no staging)

Environment	Purpose	Who uses it
Dev	Day-to-day builds, automated tests, demos, experiments. Safe to break and rebuild. Overnight shutdown OK.	Engineering
Test / UAT	Production-like dress rehearsal for stakeholder sign-off before releases. Scrubbed or synthetic data. Not for live patients / learners.	Product, clinical, compliance, sponsors reviewing a release
Production	Live Platform, Content Hub, Companion, Reports serving real users.	Ops + limited engineers; change via approved pipelines only

> **Naming rule**
> Use only **dev**, **test** (or **uat**), and **production** in docs, DNS, AWS tags, and GitHub environments. Do not reintroduce “staging.”

Why separate AWS accounts matter

Risk	Plain-language impact
Safety	A bad deploy, a deleted database, or a mis-tagged “turn everything off” script can hit production if it shares an account with Dev / Test.
Cost	Dev, Test / UAT, and Production bills stay mixed. Hard to prove savings from shutting down unused systems.
Compliance	SOC 2 / HIPAA reviews expect clear separation between production and everything else.
Speed	Teams hesitate to experiment when Dev sits next door to live traffic, and reviewers have nowhere safe to UAT.

Recommendation

Keep **Production** in the current AWS account. Create AWS account separation so **Dev** and **Test / UAT** cannot accidentally touch production. Practical options: one shared “non-production” account holding both Dev and Test / UAT stacks, or separate accounts for Dev and Test / UAT if compliance wants a harder wall. Either way, the environments remain only those three names.

Phase A
0–30 days

Decide & open account(s)

AWS Organizations, billing alerts, SSO roles (Engineer / ReadOnly / BreakGlass). Freeze tags and GitHub environments as *dev*, *test* (or *uat*), and *production* — never staging.

Phase B
30–90 days

Move Dev out of the production account

Redeploy Platform + Content Hub + Companion + Reports for *dev*. Point `devapp` DNS at the new account. Overnight shutdown for Dev only.

Phase C
Next

Stand up Test / UAT

Production-like stacks for stakeholder sign-off. Scrubbed data refresh. Same three-name model — still no staging.

03 Plan for a mobile app

What is true today

CHT is a **responsive website**. Learners and KOLs use phones in the browser. There is no native iOS or Android app in the beta scope. Authentication, sessions, catalogs, and Companion chat all assume a web browser.

Why mobile matters

- HCPs often watch short clinical clips and complete surveys between clinics — phone-first.
- Push notifications (session reminders, report ready, Companion replies) need a real app or a carefully designed PWA.
- App Store / Play Store presence builds trust for sponsored education programs.

Recommendation — three steps, not a big rewrite

Step	What we ship	Outcome
1 · Mobile web polish Now	Fix the highest-friction phone flows (login, register, watch, survey). Add “Add to Home Screen” guidance. Measure drop-off on small screens.	Better experience without an App Store review cycle.
2 · Progressive Web App (PWA) Next	Installable icon, offline shell for key pages, basic push where browsers allow. Same codebase as the website.	App-like feel; validates demand before native spend.
3 · Native shells (if needed) Later	React Native or Capacitor wrappers around the same APIs. Push, biometric unlock, store listing. Content Hub and Cognito remain the backends.	Full store presence only after usage justifies cost.

> **Decision rule**
> Do **not** fund a full native app until (a) mobile web + PWA conversion is healthy, and (b) we have a clear need that only native can meet (reliable push, offline video download, store distribution for a sponsor).

What stays the same either way

- **One login** — Amazon Cognito (already CHT-owned).
- **One catalog & analytics brain** — Content Hub.
- **One set of APIs** — Platform and Content Hub; mobile is another client, not a new product stack.

---

04 Splitting every service into app + infra repos (plus shared foundations)

What is true today

We have several product repositories (Platform, Content Hub, Companion, Reports). Inside many of them, **application code and Terraform (cloud blueprints) live together**. Shared pieces — networking ideas, Cognito patterns, "how we talk to each other" — are copied or re-agreed per repo. That worked for a small team; it will not scale as more people and services touch the system.

> **Decision (this plan)**
> We **will** split *every* service into **two repositories**: one for application code, one for that service's infrastructure — and we will add one **shared infrastructure** repository for common cloud foundations. This is the target operating model, not an optional later phase.

Problems this solves

- App teams no longer wait on infra reviews in the same pull request.
- A cloud change cannot accidentally ship with an unrelated feature.
- Clear ownership for product engineers vs. platform / DevOps.
- Easier for non-engineers to see who owns what, and what a release means.

What we want instead

- Product teams own application repos (features, APIs, UI).
- Each service owns its own deploy / infra repo (how that service runs in AWS).
- Platform / DevOps owns shared foundations everyone reuses.
- Smaller, safer changes that are easier to explain and audit.

Target repo map

Repository	What lives there	Who owns it
cht-shared-infra New	AWS Organizations / account bootstrap, shared networking patterns, Cognito standards, logging & monitoring baselines, reusable modules (ECS, secrets, queues, Service Connect). No product business logic.	Platform / DevOps
cht-platform-app	Learner & admin web app, APIs, Zoom, payments, surveys — application code only.	Platform product team

Repository	What lives there	Who owns it
cht-platform-infra	How Platform is deployed (ECS, ALB, queues, env wiring). Calls modules from shared-infra.	Platform + DevOps
cht-content-hub-app	Catalog, campaigns, KOLs, analytics warehouse, connectors (YouTube, LinkedIn, HubSpot, etc.).	Content / data team
cht-content-hub-infra	Content Hub deploy (API, sync jobs, databases, warehouse). Calls shared modules.	Content + DevOps
cht-companion-app	Chat / generate services and knowledge-base application code.	Companion team
cht-companion-infra	Companion deploy (ECS, Service Connect, Bedrock permissions, KB jobs).	Companion + DevOps
cht-reports-app	On-demand report generation service and jobs — application code.	Reports team
cht-reports-infra	Reports deploy (queues, tables, buckets, Lambdas). Calls shared modules.	Reports + DevOps
cht-video-app / -infra Next	Upload, ingest, store, stream / download of owned video files. Not metadata warehouse — that stays in Content Hub.	Video / media team
cht-media-pipeline-app / -infra Next	Transcripts, summaries, and auto-tags / topics from video (and related media). Writes results back to Content Hub as metadata.	Media / AI pipeline team

Every service — including the new video and media-pipeline services — follows the same rule from day one: **app repo + infra repo**, consuming **cht-shared-infra**.

Phase 1

Create cht-shared-infra

Move reusable Terraform modules and account-bootstrap docs. Establish the standard every service-infra repo will call.

Phase 2

Split each existing service

For Platform, Content Hub, Companion, and Reports: extract infrastructure into *-infra* repos and leave application code in *-app* repos (or rename current repos accordingly). Pipelines deploy from the infra repo using images / artifacts published by the app repo.

Phase 3

Make it the default

Document the pattern; any new service cannot land as a combined app+infra mono-repo. Shared modules remain the single source of truth for “the CHT way.”

For non-technical readers

Think of **shared infrastructure** as the building's plumbing and electrical standards. Each service is a floor with **two rooms**: one for the furniture (the application) and one for that floor's utilities panel (how it is wired in AWS). Floors should not invent their own fire codes — and furniture changes should never require rewiring the whole building.

05 Video services: upload / ingest, transcripts & tags, Content Hub analytics

What is true today

- Most public clips and podcasts play through **YouTube** embeds.
- **Google Analytics (GA4)** records views, video start / progress / complete, and share clicks in the browser. Those numbers live in Google, not in our databases.
- **Content Hub** already stores campaign and catalog **metadata** and can hold per-platform **analytics** (YouTube views, LinkedIn ads, HubSpot snapshots). That remains its job.
- Zoom session recordings live in Platform storage for admin use; that is different from a learner-facing owned-video library.

The gap

We do not own the upload → store → stream / download path for catalog video, and we do not have a dedicated pipeline that turns video into transcripts, summaries, and tags. Without those, we cannot fully control the player, branding, or **platform analytics** — and Content Hub cannot become the complete metadata + analytics home for our own files.

Target picture — three clear jobs

Service	What it does	What it does *not* do
Content Hub	**Metadata and analytics.** Catalog records (title, KOLs, topics, campaign, rights, pointers to files), performance warehouse (views, completion, campaign metrics), and what Reports / Companion are allowed to read.	Does not store large video files or run the upload / encode pipeline.
Video service Next	**Upload, ingest, store, stream, and download.** Accepts files from editors / ops, processes them for playback, serves secure watch / download URLs to Platform (and later mobile).	Does not own the catalog of record or the analytics warehouse — it tells Content Hub where the file lives.
Media pipeline Next	**Transcripts, summaries, and tags.** Listens for new (or updated) video, runs speech-to-text, produces summaries and topic / tag suggestions, writes those results back to Content Hub as metadata.	Does not host the player or collect click / watch analytics.

> **For non-technical readers**
>
> Content Hub is the **library card catalog and the scoreboard**. The video service is the **vault and the projector**. The media pipeline is the **clerk who watches each new tape and writes the transcript, a short summary, and the subject tags on the card**. Platform is the **theater** where learners sit — and where we can finally measure how the show went.

Why self-hosting unlocks platform analytics

When video plays through **our** video service (on our domain, under our control), Platform can reliably capture first-party events: play, pause, progress milestones, complete, share, and which clip / program / campaign the learner came from. Those events become **platform analytics** we own — not a best-effort read of YouTube or GA4. The same events are then stored in Content Hub so official reports stay consistent.

Platform analytics (once we self-host)

- Watch funnels per clip, playlist, program, and campaign.
- Completion and drop-off curves we define (e.g. 25% / 50% / 75% / 95%).
- Share and chapter clicks tied to logged-in users (where allowed).
- Admin dashboards that do not depend on Google's UI.
- Clean handoff of the same events into Content Hub.

What Content Hub then holds

- Catalog metadata (including tags and summaries from the media pipeline).
- Pointers to files in the video service (not the heavy files themselves).
- Transcripts (or links to them) for search, Companion, and reports.
- Performance analytics warehouse for campaigns and clips.

How a new video moves through the system

1. Editor / ops **uploads** a file to the **video service**.
2. Video service **ingests** it (validate, encode / package, store) and is ready to **stream or download**.
3. Video service registers the asset with **Content Hub** (metadata stub + storage pointer).
4. **Media pipeline** picks up the new asset → transcript → summary → tags / topics → writes those fields back to Content Hub.
5. **Platform** (and later mobile) lists the clip from Content Hub and plays / downloads via the video service.
6. Watch and share events flow: Platform analytics → Content Hub warehouse → Reports / Companion.

Self-hosting options (decision later, direction now)

Approach	In plain terms	Fit
Managed video platform	Specialist (e.g. Mux, Cloudflare Stream, AWS Media) encodes and streams; our video service wraps upload, auth, and URLs.	Fastest reliable path; pay per usage.
DIY on AWS	S3 + CloudFront + our player; encoding pipeline we operate inside the video service.	More control, more ops ownership.

Approach	In plain terms	Fit
Hybrid	Keep YouTube for discovery / SEO where sponsors want it; owned clinical library goes through our video service first — with platform analytics on that library.	Pragmatic transition.

Phase 1

Define contracts

Content Hub = metadata + analytics only. Agree IDs, what “a view / complete” means, and how the video service and media pipeline register results with Hub.

Phase 2

Video service (upload / ingest / download)

New app + infra repos. Pilot a small set of clips; Platform plays from our URLs; turn on first-party **platform analytics**

.

Phase 3

Media pipeline (transcript / summary / tags)

New app + infra repos. Auto-enrich Content Hub metadata so search, Companion, and reports can use transcripts and tags.

Phase 4

Scale & hybrid

Expand owned library; keep YouTube only where marketing still needs it. Platform analytics and Hub reporting stay aligned.

06 Lower cost, more autonomy, stronger products

The account split, repo split, and clearer service boundaries are not only about neatness. They are how we **spend less on idle infrastructure** and how we give teams the **control to build serious products** for Community Health Media — without stepping on each other or on Production.

How cost decreases: scale each environment to what it needs

Today, Dev and Production share one AWS account, so it is hard to turn “test” systems down aggressively without fear of touching live. With **Dev**, **Test / UAT**, and **Production** clearly separated, we can right-size each one:

Environment	How we scale it	Cost effect
Dev	Smaller databases and fewer running services. Overnight and weekend **scale-down / shut-off** when no one is working. Rebuild cheaply when needed.	Largest savings opportunity — Dev should not look like Production 24/7.
Test / UAT	On when a release is being reviewed; scaled down or off between cycles. Production-like shape, not Production-like spend every hour of the month.	Pay for dress rehearsal when we need it — not idle UAT all month.
Production	Sized for real users and reliability. Scale **up** with demand; avoid over-buying “just in case” because Dev/Test are no longer mixed into the same bill and risk.	Spend where learners and sponsors feel it; stop subsidizing unused test capacity.

Concrete levers

- Fewer running containers in Dev / Test when idle.
- Stop or pause non-production databases overnight.
- Right-size each *service* (Platform, Hub, Companion, Reports, video) independently — no "one size for the whole account."
- Clear bills per environment and per service → budgets and alerts that mean something.

Why this only works after we separate

- Shared accounts make "turn it off" dangerous.
- Combined app + infra repos make ownership fuzzy — nobody owns the waste.
- Separate accounts + per-service infra repos make scale-down a normal, safe habit.

> **Plain-language takeaway**
> We stop paying Production prices for workshop and dress-rehearsal rooms. Each environment gets only the power it needs, when it needs it.

How teams get more autonomy and control

Splitting application and infrastructure per service — with shared foundations everyone reuses — changes how people work:

Today	With this model
Feature work and cloud changes compete in the same pull request.	Product teams ship application changes; infra repos change on their own cadence with clear review.
One shared account means everyone is afraid to experiment.	Dev (and Test / UAT) are safe places to try, break, and learn — Production stays protected.
Ownership is fuzzy across Platform / Hub / Companion / Reports.	Each service team owns its app and its deploy. Shared-infra owns the standards, not every ticket.
New capabilities (video, media pipeline, mobile) risk becoming "another folder in the mono-repo."	New products start as first-class services with their own repos and cost line — easier to fund, staff, and measure.

Ability to build robust technology products for the company

Autonomy without standards creates chaos. This plan pairs freedom with a shared foundation so teams can build **robust** products:

- **Clear product boundaries** — Platform for the learner experience; Content Hub for metadata and analytics; video for files; media pipeline for transcripts / summaries / tags; Reports and Companion as consumers of Hub.
- **Reusable "CHT way"** — networking, login, logging, and deploy patterns live in shared-infra so every new service starts strong instead of reinventing plumbing.
- **Safer change** — smaller repos, smaller blast radius, environment-appropriate scale → fewer production incidents and faster recovery.

- **Room to innovate** — mobile, self-hosted video, and platform analytics become deliberate products we can invest in, not side projects squeezed into a shared pile.
- **Accountability** — each team can see its own cost, its own uptime, and its own roadmap — which is how a technology organization scales with the company's ambition.

> **For leadership**
> This is an investment in **operating leverage**: lower ongoing cloud waste, clearer ownership, and a structure where CHT can keep shipping durable products — not only features — as Community Health Media grows.

07 Suggested sequence & decisions needed

Do these in order so each step makes the next safer:

1. **Approve AWS account separation for Dev and Test / UAT** — keep Production isolated; never reintroduce staging.
2. **Stand up cht-shared-infra, then split every service into app + infra repos** — this is the committed operating model.
3. **Turn on per-environment scale-down** — Dev (and Test / UAT between cycles) should not run at Production size 24/7.
4. **Mobile web + PWA path** — serve phone users without a native bet yet.
5. **Video service** — upload, ingest, stream / download; unlocks platform analytics on our player.
6. **Media pipeline** — transcripts, summaries, tags written into Content Hub metadata.
7. **Content Hub stays metadata + analytics** — official numbers for Reports and Companion.
8. **Native mobile (only if needed)** — after web/PWA demand is proven.

Decision for leadership	Options	Recommendation
AWS accounts / envs?	Stay shared · Separate Dev · Dev + Test / UAT + Production	**Three environments only**: Dev, Test / UAT, Production (no staging). Separate accounts so Production cannot be hit by mistake.
Repo structure?	Keep combined · Shared modules only · Full app + infra per service	**Full split**: shared-infra + app repo and infra repo for every service (including video and media pipeline).
Cost posture?	Leave running 24/7 · Ad-hoc cuts · Scale per environment by design	**Scale Dev / Test down by default**; Production sized for real demand; budgets per env and per service.
Mobile	Web only · PWA · Native now	**Polish web → PWA**; native only with clear demand.
Video architecture	YouTube only · Hub does everything · Separate video + pipeline + Hub	**Separate video service + media pipeline**; Content Hub = metadata & analytics only; Platform analytics on our player.

Bottom line

Keep only three environments — **Dev**, **Test / UAT**, and **Production** — with Production isolated. Split every service into app + infra repos on shared foundations. **Scale each environment down to what it needs** so cloud cost falls. Give teams clear ownership so they can build robust products with autonomy — video, media pipeline, mobile, and analytics included — while Content Hub stays the home for **metadata and analytics**.

Related internal materials: CHT Platform Architecture, Data Dictionary, Data Flow, Beta PRD, Platform Assessment Report. This brief intentionally avoids implementation tickets; engineering can produce detailed specs once the decisions above are approved.